ActInf OrgStream 009.1 ~ The Value of Open Source Software (Hoffmann, Nagle, Zhou)

Source: YouTube **Playlist**: Active Inference OrgStream **Duration**: 1:07:50 |
Uploaded: Unknown **Transcript method**: auto_caption

Hello, welcome. It is June 18th, 2025 and but your cover slide had me guessing for sure and we're in active or extreme 9.1 with Manuel Hoffman and with Bradley Alysia as guest. We will be discussing their recent paper the value of open-source software and looking forward to this a lot to say that I'm sure will come after your presentation. So thank you again for joining and to you for the presentation. Great. Yeah. Uh fantastic. Thank you Daniel for inviting me to present this work and yeah Bradley looking forward to the discussion and everybody else on YouTube as well. If you have any comments, obviously that's uh much appreciated. All right. Well, so let me just dive right into this, right? Um so we're covering here um the value of open-source software. Um as uh let me just give you an idea, right? And probably um you are closer to that than maybe some of the other audience member to whom I usually present. But um you can think about a bunch of companies here, Google, Ford, Intel, John Deere and we usually can just ask you know what do they have in common? Well actually what they have in common is that among others they are really examples of um users and contributors to open source software and um we know that open source software is a um really a cornerstone of modern software development. So 96% of firm code bases contain some uh form of open source software. That's according um to the software analytics uh company synopsis. And 76% of all code infirm code bases is actually um open source software code. It's um hugely important in cutting edge technologies, artificial intelligence, quantu quantum computing, big data and analytics and uh many other areas. So um one um venture capitalist Mark Anderson once said software is eating the world and that was enhanced by Joseph Js who um used that saying software open source is actually eating software faster than software is eating the world. Um just talking about the dominance of open source software um which uh just uh to give you the rough idea of the definition right open source software is software whose um source code is publicly available usually under some permissive uh license in particular um we are interested in free open source software very often um which is a public good um it's

generally non-rival so if I consume it there's not less available um for you and it's usually non-excludable. So um basically it's the ideal form of a public good um that everybody can consume and um often you can really think of that as a as a global public good like education um individuals and firm have used firms have used open source software and contribute to it substantially right so there has been a bunch of literature especially uh by by my co-author here uh Frank Nigel um but also a lot of other people and we are continuously working on better understanding um the open source software ecosystem, companies and individuals that engage in it. Um so now you're probably very familiar with this uh slide, maybe I've seen it too often, this kind of figure, right? um where we can see this huge machinery of our modern digital infrastructure and then there is this little piece at the bottom which is a project by some random person in Nebraska that has been thanklessly maintained since 2003. So uh what that tells you right there is um first there is some huge dependency on some highly active individuals some open- source maintainers um without um our without um there could be a potential collapse of our modern digital infrastructure and on top of that there's the issue of the tragedy of the uh commons which is um that which is common to the greatest number gets the least amount of care according to aristotally right So there's some kind of free riding uh potential not only but also I think um issues that we observe anecdotally and which we also see here in the data um as well. So now um why do we care about the value of open source software? Well, I guess we care about the value of a lot of goods. Um but here in particular we do not know um actually what's the value of open source software because it doesn't even come up in traditional uh gross domestic product measures right that are in accounting systems and um also some firms may just not be aware of productivity and cost-saving benefits. So that's why we um you know are bringing out this paper to uh better understand the value of open source software measuring um the value here in a very simple way um as the price um times quantity P * Q and so here opensource software you have really the problem the price of open source software is zero right um there is no price that is um the actual measurement problem and the quantity well it's infinitely replicable um because the license allows for free copying and usage is not easily trackable. So we have a problem on the price side, we have a problem on the quantity side and we're not saying that we're going to solve this problem, but we're um saying we're going to we're trying to make some inroads here. So um this in the spirit of what Warren Buffett said, price is what you pay and value is what you get. So right we price could be one idea of value but it's really not the actual value and um of course we want to provide a value measure here because um usually um a lot of forks managerial folks

firms working in companies but but really anybody wants to understand um it wants to get an idea of how valuable something is. So um because otherwise um people may not um allocate resources towards it or do not really understand the importance of it. So we ask how much would we lose really if open-source software did not exist or ask differently how much would it cost us to replicate the universe of open-source code. Um, and for prices, we're going to calculate uh the prices using what we like to call a replacement value or so-called um you could also use a purchase price. And for quantity, we will use two large data sets. The largest ones that we are aware of um one is the census of open source software which is um inward facing towards the within the company the products that they are using and then um reselling and then built with which is an outward facing data set um that is um basically based it's based on websites and the usage of of open source software and website. Now to measure the value of open source software you could um or really the value of any um anything that is difficult to measure you could go about it in two ways. You could go about it from the output market side. Uh the output market would be the the goods market from the firm's perspective and you can just um let's say you have a good that has a a price of zero. Then you find a substitute good um that actually has a positive price and uh you may be able to impute that to get an understanding of um what the value may be. and North House have famously made this very generary case for greenhouse gas emissions and um Greenstein Nagel have for example calculated the value of Apache um servers being around $2 billion and that has been extended um with web servers in general the value being around $4.5 billion and they could do that because they have had some information on what the substitute good was and so so that is very helpful right um so they have information on usage of all of the web servers but then we can only really look at one particular type of um software here web servers and it's difficult to expand that because you would need to find a basket of goods a gigantic basket for each of the open source software packages and some are not um you know are really small pieces some are larger pieces it's difficult to find those um substitut goods. Um so what we did is um we're going to use the second approach, the input market approach um or the labor approach. So from a film's perspective, you're going on the labor market and we're going to look at the replacement value via labor cost. You can think about that in the spirit of um household innovations, bicycle and hipple. Um main idea here really is um let's think about the housework that you are doing, right? you're you're cleaning um something, you're vacuum vacuuming in your house and you're not getting paid for that, right? You just do that in your free time. That's not going to add to GDP. But you could instead hire somebody and they could

um do the work and vacuum for you and that would be the price that it would cost um to do to engage in this household um activity and then you could measure the value and um we are going to do a similar exercise and we're not the first one here there's uh blind and co-authors that have done a kind of similar exercise um for the European Union um Robins uh and cos have done an exercise for the United States where they estimated the supply side value being around1 billion to 38 billion. We do this um not on the European or US level but we think about that as as a global estimate but we are going to cut down and we're going to look at firm specific contributions. So we're going to discard kind of the long tales of contributions that may um happen um from from individual small contributions. So that's where our estimates may differ. So now the good thing about this approach is we have all of open source um we can use all of open source software and we can assign some kind of price by imputing the wage that it would um cost to create this um product. And the negative aspect is maybe it's difficult to stay to to get the usage information. But now um we're going to um we are also approximating um the usage information through our two large data set. So now um for our methodology we're using really multiple counterfactuals. Um well we are mainly using method two but uh in one counterfactual we're also um going to use um a part of method one a little bit and I'm going to explain this in more detail here. So our first counterfactual is uh the supply side counterfactual. So it's the idea that open-source software as a concept just purely doesn't exist and now the whole code has to be rewritten once. Now um we're doing that by um looking at all open source packages that are used by companies uh in particular. Um we uh think about addressing production externalities. So um like we are just holding constant people that are working one company are not going to move to another company and are produ or like are not benefiting from the production of one piece of code when they are then uh producing another piece of code. um but they are just producing those pieces of code separately. Then um we count the lines of codes per package and calculate the labor replacement um value um so which is for us um in this case the value of uh from the supply side and we can um dive into a little bit um how this labor replacement cost value works exactly um after going through those three uh counterfactuals. Now the second counterfactual is um well if uh the concept of open source doesn't exist anymore. Now what would happen is maybe one firm would rewrite would would emerge and rewrite the code and then sell it to all of the other firms. So um that's now we would just think about obtaining the information usage information that we have from both data set on the packages we address uh consumption externalities holding so there are no additional kind of spillovers then we sell um the the one seller um gen creates all of those

packages and then they sell it at the market price P. So here you can really see that it's just um the price times quantity over all of the packages minus the fixed cost that it um takes to generate um those packages which is really the value from the supply side um from the first counterfactual and the elasticity of demand here is around one. We did some um that we set around one um based on the literature but we um estimated it and varied um it so engaged in some robustness checks that you can find in the papers as well paper as well. And so the main idea is that we find an optimal price P here given all of the quantities and then we can um create uh then we can get a measure of the overall value from a one seller demand side. Now a third counterfactual is what we like to call the firm auto key demand side and um this concept of open source software again doesn't exist anymore. Now each firm has to write rewrite the code by themselves. Um we again obtain all of the usage information address consumption externalities but now there's a single replacement idea within a firm. So when you use um when you use a product yes you can use it multiple time a package but we are really thinking of that um we don't have to generate that there's no additional value that is being generated from using it additional times because you're just using you can just um freely move it within the company and um there's no price for internally using it. Now we calculate the value here by using the supply side value and multiplying it by all the times that it has been used each package um for each of those companies. All right. So um what is the data about? Well, we have the uh Census 2 of free and open source software which has been um a really a mega project I would say by my uh co-author Frank Nagel at um HPS. Um and this is uh from uh 2020 data from 2020 in particular. There is a third version um coming up and there are continuously new version and we're trying to see how we can get more information about open source software usage that is particularly firm related and here you can see uh code usage of firms globally. So there are around 600,000 packages but we are going to really uh home into the top packages. So we're cutting off a huge tail um on on the right side. Um this is information from software composition analysis firms. So they are scans. What those companies um here Synopsis and Sneak for example or FOSA are doing is they're usually asked um to scan um all of the usage of um open source software and software in general within firms. For example, when a merger is happening um of firms or before a merger is happening because the companies want to know what kind of potential liability exists and um so they really check for license compliance. Now um a second data. So this is the kind of the inward facing um data set um that um we construct and the second complimentary data set is from built with which um is a um service which basically scans all of the websites um globally

and gets the code from all of them and then um it measures all of the technologies that are embedded in those websites and among others. this cont contains um all open source software web development and we try to approximate that we think here an underestimate we are just basically fing focusing on JavaScript um to make it easy on us but I'm sure um there would be uh you know like there's an even larger value here those are 8.8 8 million websites. We wanted to home into the ones that are distinct from websites. So we match this with a a couple of uh company databases that allows us to identify companies. So like compost and pitchbook which gives us around 3.3 million websites for um almost the entire year of 2020. Now we have those two complimentary data sets. There are internal faith internally inward and outward facing and um additionally for um for an analysis on on inequality and what the contributor what the developers contribute as a value we uh use GitHub. we um obtain the share of contributions to a project um from developers. Um so really the commits to a project and the number of project contributions and um as the fourth data set that has been um relatively recently added are firm contributions. So we try to obtain an understanding of firm affiliations of projects through the emails of the committers. So if you used an email that is um tied to a company that we can identify in August compost or pitchbook where um or is this a global database on companies and compost is um for for public firms and pitchbook for for small for smaller entrepreneurial firms. um then we would attribute that to be a firm affiliated project and um we get the share of commits that are uh firm affiliated per project. Now um I mentioned we are using this kind of replacement value labor cost approach. So we are really using the constructive cost model also called Cooko from the USD department of defense to estimate software project cost. It's quite a flexible model but we just we are going ahead with the default parameters but again we're engaging in some robustness checked with those parameters as well. So really here it's here the effort stands for person months effort. So how many people do you need per month and then LOC stands for the lines of codes and thousands of lines of codes. And we just have uh some parameters here that we um that are nonlinear transformation that we uh that we can vary but we use a default um for this exercise. And um one aspect is well once we know you know how many people we need um then we also need to provide give an idea about what the wage for those individuals may be right and so from salary expert another database we got uh the monthly gross salary uh with the range from 3,000 to$10,000 um dollars um so from a low and high wage country. Let's see if this button works properly. Um so here you can see the top 30 country countries that are included um for what we call the the global wage. Um so we included 80% 88% of GitHub activity from 2020

and we used all of those company in the global in a global wage estimate. So we really have three estimates. One is um you like um an estimate where we used um a low wage from India, one where we used a high wage from the US and something in between with all of those countries that I just show you as a as a global average wage. All right. Now, to give you a sense of the data, here are some uh descriptive statistics um for both panel A, which is uh the sensors and built with data in um panel B. So outward and or inward and outward facing. So you can see the lines of codes for all packages. In particular here the number of pack like yeah the lines of codes is roughly um maybe 70% of that um when we just focus on the top five languages and we can see um the usage of all packages around two 2.7 million times that packages were used here. the um around 1,800 or packages or for the top five languages around 1,600 packages. Build has a little bit has also roughly the same ratio of top five languages that we identified based on um GitHub's uh top five languages um during that time period. And I show you what those um where um when I go through the results um but those are around 740 projects. All right. So now um the comes here's basically the main finding right if you want to take um anything away from this it's um it's here the value of open source based on pure code so we're getting rid of any kind of addition what could be uh data for example or could be m maybe machine generated and um what we find is for a low wage country would be the Indian wage we found around $1 billion value on supply side of firm related open-source software um packages where there's a global estimated around 4 billion and six billion on the higher side for US wage um as a value. Now, um when you have one seller that is taking and that is recreating everything and then selling the packages to those different um firms that are using it, we have almost we have basically a constant value of 5 around five trillion dollars. Um there is some variation but basically the wages here don't matter as much because they're in the fixed cost. Um and then finally when we assume that all of those companies have to replace um open source software packages by themselves internally and they don't uh resell sell products to other firms. Then we get a value of around um $88 trillion um globally um in this case. Right? So you can see this look at this as some kind of upper bound and maybe we may be in between this one seller case and the firm auto key case in reality. Now um well let's take a the 5.25 25 trillion as kind of a lower bound estimate, right? Um you accounting in particular for usage. So not just the uh supply side of your availability of packages. And so this is around right 5.25 trillion in demand side value over the time. Um in particular measured in the year 2020. And we know that private firms spent around 45% of um software uh cost on prepackaged software um so proprietary software right so now uh let's

assume this kind of spending that is actually US estimate holds globally um this would be around $2.36 trillion um of proprietary um spending on on closed software if you like now you add the value of the 5.25 25 trillion to that it's around $7.61 trillion just purely in software cost. So what that implies is that firms sp firms now would spend 3.2 times more on software than what they currently are spending um in relation to uh because open source software the concept just purely doesn't exist anymore in this um world, right? Um so that's a huge um kind of costsaving for them um when you just think about open source um software being freely available. Now we know there's a substantial value. So one question that is quite natural here is how evenly is this actually distributed? Well, and here are some examples of um well, this is here the top five languages plus um Go um from uh from GitHub. And you can see um this is a supply side value of just recreating um all of open source software once um of the top five languages. There's some substantial variation where Go and Java um you know have quite a high value. Um, Python for example has a relatively low value. Type code JavaScript also relatively low value on the supply side. Um, see somewhere in the middle. Now um this is not surprising. This is more of a robustness uh check um on the supply side for so this was a census data set. the inward facing, the outward facing build with data set. We conditioned already on JavaScript which is often also related to TypeScript and so we picked up B basically all of that variation um in in the externally facing data set. Now for the sensors of open source software unfortunately we don't have any information on the hetrogenity by industry. So we try to um think about that a little bit more in the context of um the outward facing build data set and we got some we could show you some hetrogenity in the demand side value of the trillions of dollars. So we can see um here in administrative and support and waste is quite high the value healthcare and social assistance professional scientific services other services quite high retail trade also relatively high or you see um manufacturing interestingly it's somewhere somewhere in the in the middle I would say um management of companies and enterprising mining and public administration utilities or agriculture is um the value is relatively uh limited it and to some degree that may be just of uh lower uh yeah just substantially lower usage in those industries. Now um how is this value distributed or or I guess where does this value come from? Um when we think about the developers here and we plotted a Lawrence uh curve. So with this idea of a genie coefficient of being one right um if you have this if you think about the Lawrence curve here as tracing the 45 degree line this is the um share of the value on the supply side. This is um the um y-axis on the x-axis you have the cumulative share of developers that are creating this value and they're creating um they

engaged in uh repositories. Um here the value is in blue and the engagement repositories in uh orange. Now if this blue line would be exactly aligning with the 45°ree line that would mean that every developer contributes evenly contributes equally really to uh the value share but we don't see that. We see that the um that this blue line here that is the value line for the share of developers is actually very close to the corner here and very far away from the 45 degree line and that's what we would expect also from the literature right um that there is a share of developers that is um here maybe around um five or 10% of individuals that are creating a substantial value. They are generating most of the value here on the suppliers that are creating the packages. And interesting so um interesting part here is that they are not just active in a few projects but they're active here in yellow in in many many different projects on average. Now um we could go to the one seller demand site. We actually see a very similar picture here. um the we see a huge inequality of just a few um developers contributing a substantial amount to the value share and we see this again um when we look at the firm auto key case so now um to conclude here from um the inequality story 5% of open source software developers will liquidate 93% of the supply side value and as a bound 99% of the firm auto demand side value. Um so just the 5% of developers create a huge amount of value um on the supply and the demand side. Now what about the firm uh the value that firms uh that is kind of firm affiliated right that potentially firms are creating. Well um what we find is that 23% of projects are firm affiliated. Well um that implies that 77% are not. Um now firm contributions to the value of open source software uh look as follows. We see the share is around 6% of the firm contribution to on the supply side on the one seller and firm auto key it's between 7 and 2.8%. So the firm contributions here are as measured by the thermal radiation which may be an underestimate here right but it's um actually quite low. The interesting aspect is that they supply more than they demand based on those um usage statistics. Um also very interesting if you think about um you know individuals here certainly create a lot of value in this space but uh firms are also doing valuable things when you just uh think about correlations of uh film affiliated uh repositories. Um here in uh like reddish and then in kind of like this blue color you see non-firm affiliated um the the shape the density distribution um for the t days from the last commit measure which implies um here that firms are really um substantially more active. the distribution is very close to relatively more close to zero than for individuals and that may imply that firms projects are just more actively maintained and that is overall a good thing. Now um there are a bunch of reasons why we may think that um what we're finding here is the value of open source software uh that we actually underestimating the value. Um the global

data that we have is very particular global data uh from those software composition analysis firms which is as good as um what we could get as data but but the you know global data is still incomplete. We cut off the tail of the distribution as mentioned. We also got rid of comments, documentation, empty lines, anything that is not pure code. Um, we have an additional estimate for that in the paper. Um, also you can think about code being improved over multiple versions, right? We are not accounting for that and that is but that is also costly. And you could also imagine well there are a bunch of um projects that a company has to um work on and not all of the projects may work out but so it may be a necessary process of to have a lot of project failure with one successful project but that implies that there are also costs that are associated with those kind of failures and those costs are not um included here. Now um we also think that even the highest wage scenario is kind of a lower bound on wages because if you think about the individuals that some of the individuals that are active in this space they can obtain um especially highly skilled individuals in open source they can um obtain quite high wages u from some of the um big tech companies. Now further we just don't have the Linux kernel in here which is probably um yeah a huge uh value driver u when it comes to open source software. Now to conclude the value of open source software on the supply side is around $4 billion with a one seller demand side value of 5 trillion which can go up to $80 trillion when firms are um just recreating codes by themselves. We find hetrogenity across industries and developers where 5% of develop opensource software developers create around at least 93% of the value and um firms here make up really a small share but projects are a bit more actively maintained um by or projects by firms are more actively maintained than by individuals. So without open source software there would be really a large loss in value. Firms would need to spend 3.2 two times more on software than they currently do. Oh, thank you so much. Thank you. Awesome. Lot there. Big iceberg. Only small amount showing, right? Perhaps first uh Bradley maybe introduce and and coming from the open-source sides that you've worked on like where does this take you? what does this reflect on you? Yeah, so um you know I I come from a bit of a different perspective. I I do a lot of open-source uh education and and project management and then I've worked in open source not in in terms of how corporations interface with it so much but how basically you can do scientific research with open source and other types of things like you know creating games with open source and I liked what you had here. I think that was a very good analysis of looking at like sort of putting a value on you know what we have out there and um but I I I wonder especially with respect to like the active inference aspect you know is it that people are motivated by

money or is it that there in some cases they're motivated by sort of a need in the in the communities they're working in and in the sort of the you know cuz a lot of like if we think about Linux and some of the things that have been created there it was largely because there was some utility of a tool that they needed and there wasn't a tool anywhere else. So they created the tool and then they made it into this thing that was distributed and then it went from there. So the original motivation was actually more kind of like almost like maybe reinforcement learning or something like that. And then you know when you get into when once it's distributed then you have this uh financial imperative that might take over or this sort of um cost imperative like cuz I know you showed that figure of the maintainers and how they play this critical role in open-source projects and and you know a lot of the problem there stems from people not have you know kind of doing it on the side and then not having enough time to devote to actually maintaining these large packs. packages. So, you have these vulnerabilities cuz people just don't have, you know, it may start that they love the project and they're motivated by self-interest, but then over time you get, you know, a project that's so big that maintainers just simply can't afford to do that, you know, to spend like 70 hours a week maintaining this project. So, I mean, you know, I wonder about that. Like, what are your thoughts about different types of motivations for open-source um production? Yeah. No, I think you're absolutely right. um like the motivations um especially at you know like the origins of open source and I think even what is created right now as open source software tools is very often yes there we need some kind of tool we don't have it and so let's create it and um I guess that's the origin of a lot of innovations just generally speaking um like or that's one of the idea of innovations right like um I I don't have this particular thing I a need and now well if nobody else is doing this I'm going to do it but they're willing to also share this you know like the open source community um to share uh things openly that's the whole spirit of it so yeah I think um non-punary benefit like non-picurinary motives are a huge driver of what is going on how open source software exists in the first place right and um this idea of sharing and supporting a potential even like reputation building. And there are some you know aspects of course where this then like like it's also kind kind of like a skill building exercise but maybe people don't do that necessarily just for that purpose but it comes with it. Then the like once you create those kind of projects and are really successful in it as you point out then maybe you don't have the time anymore to even uh be able to maintain those uh projects. uh just for free. Um but then you have learned a substantial amount on the way which uh is quite is valued quite highly in the marketplace. So companies I think do value that and we have seen

papers around that where where people even try to signal um hey like I'm I'm contributing to you know like open source by engaging in a couple of commits before I'm getting hired and then after that you can see kind of like a drop off in um the engagement. Um but um I think yeah there are many different motives but certainly the nonpunary motives is um quite a critical one in this environment. I completely agree. Yeah. So I mean the other Oh, go ahead. Go ahead. Oh yeah. So the other thing I wanted to mention is that like you know the way kind of we do open source is kind of dependent on certain tools that we have. So you know we have forks of repositories and you know that makes it easy for people to say copy a repository and make modify it and distribute it and it's really interesting. I wonder if you know people thought about like the economic sort of benefits of a lot of the innovations of uh version control where you have like forking and you have like you know being able to control different versions and how they're distributed because that that kind of plays a key in like how people get value out of it. If I see a project I want to fork it. I want to create like I want to basically add value to it, change it so that it's useful to my field or my group and then you know if a thousand people do that that's a lot of value out there and if you can't do that easily then there the value is limited because it makes it harder to sort of keep all the distributions maintained and keep all them all consistent. So, you know, there are all these tools that kind of showed, you know, were kind of before we had the kind of version control we have now. It was probably much harder to get value out of the open source software. It was just kind of like people kind of distributing it maybe on discs or, you know, there were earlier versions of version control. And it would be very interesting to see where that sort of started up if that if there were like the set of innovations that we have say with GitHub played a huge role in kind of like kickstarting a lot of the value of open source. Um yeah that's a great point and we are um in constant communication with the guys at GitHub and have some other work with uh the people at GitHub and and generally yeah I agree with that also um like the benefits of version control I would think are enormous I mean they just contribute to this I like to outsourcing in general right um but but I I haven't seen a paper on that so because I I guess it's difficult Yes, we can look into the past uh and there will be we're getting value out of it. Okay. Sorry. Um I think you're a little bit frozen. Okay. Um now you're back. Yeah. Yeah. Okay. Um yeah. So just right. So you mentioned this point of disks um you know like code being distributed on a disk versus now with the later version controls the ones where everybody can access it. And I think this would be uh yeah it would be very useful to understand the world where we just have disks versus the world where we just have where we have the version control that we

have now but have those two worlds right next to each other. And it's it's as difficult as our exercise that we um did in this paper because we have to imagine different kind of counterfactual worlds and then try to impose some assumptions. I'm sure this can be done or maybe there's a clever way of of of getting an understanding of what you know the benefit of version control is. I'm sure that just the pure number of people that are getting pulled in right you can you're just building on on you know pulling in the crowd substantially more and making it more convenient for everybody to contribute in a very easy way and that's why open source software is I think also successful because um of yeah crowdsourcing and um yeah you're just very easily there's a there's zero cost basically from you replic vicating a package. Yeah, just to this last point, we're still very generationally early in the global internet facilitated exchange of open-source software. So it's kind of like looking at the early developmental phases and and there's a lot of core technologies like Git and services like GitHub and there may be other facilitating technologies to open source overall. Um some that people are probably working on like ways for payment channels or or credit assignment to propagate back um upstream uh different ways to think about the um the value and security of it. Like the part about the paper and the presentation that that um was most interesting to me was these different economically grounded estimates which differed by orders of magnitude ultimately. So it was a very restricted and humble analysis because and then because it didn't include Linux, it included some packages on some services from some companies. And so it's kind of like turning on the flashlight to use economically normal normative estimation methods, different ways of triangulating what just the economic narrow value like software with a price tag would be let alone this training improvement of infrastructural type software. So just just that even when so restricted still there was a wide range of estimation. So I guess how how does estimation move forward and then what do those estimates who do you tell about those estimates or who who who do you who do you want to see do what if they were to know that a given method of value assessment were this or that? Mhm. No, this a great point. I think part of um I think there are some efforts in um yeah in government and um when it comes you know to estimating GDP in general and we would love to see um an a version um brought into the GDP right where you can estimate the where the value actually impacts um GDP measures that would be fantastic of course this doesn't mean that um what we have used here needs to um be incorporated. One has to really um think about what's you know like an optimal version for that. But also just generally speaking I think um provide I think providing those different estimates is helpful for um first providing one estimate I think is helpful when

you talk to anybody that has not even an idea has not even heard about open-source software right there. Some people may just go on the internet, they use some code and they're happy that they found it and they are, you know, very new to the space and you can um you can pull people in with hey, you know, this is a giant space um and it's an extremely valuable space. So that's one even when you look at the lower bound estimate I think um I like to think about that as I had some work on vaccination prior to the pandemic and I had to explain um you know to people the importance of that and I feel this is kind of very similar for open source software when you talk to um people that are not in the space they are not going to appreciate it as much and even some of the people that are they just um you know like use it there's a lot of free writing going on which is okay but um there's also some aspect of you know like there's a lot of value created and how can this be sustained and yeah so I think informing everybody first about the value is important it's also important um not just for society as a whole but also when you think about companies in particular managers um there are some organizations that have actual like open-source software units maybe uh and developers are using open source software tools but then um when they're engaging in their work um they have to also justify to their managers why are they doing what they're doing right and what kind of value they are creating and if they can't do that it's kind of uh challenging to talk to their manager the manager will be like I don't know what you did um but I know exactly when we buy a product from Microsoft that cost me x dollars and um yeah so and I don't know that you know like what's what's the cost when we do this here with an open source software tool and finding the right comparison and I think that's where we should be going um so that um yeah open source so that people can justify the usage of it within companies even more I think that would be great if if this paper can just make a little bit of a dent here um in the general perception and perceptions at the workplace around open source software as well. Yeah. Yeah. I think that quote that you had there the price is what you pay value is what you get and it's the value of open source software and then there's like this deeper psychology to paying or valuing what has been provided and the different sensitivity to losing what already is there as opposed to gaining what could be and all these different types of costs like time cost tech burden those things come up in the industry setting and in the research setting. So again, it's just sort of like opening up some paths to explore. Well, if every firm had to redevelop just some JavaScript packages, we'd be talking about like the economic activity of the whole world. So we know we can't do that. So okay, so we're going to talk then we're going to have some sharing of code. So then what does that look like in in some sort of extreme world where every firm

develops every package and every operating system and like even the largest uh corporations can't develop a secure Linux kernel to there's one version of everything. Somehow everyone knows how to find it. There's one package for every single thing. Um where are we? Where do we want to be? Where is the redundancy contributing to resiliency and to to maintaining portfolios of different approaches even if there's like multiple visualization packages for a given language? It's sort of like it's not necessarily wasteful, but just to be able to assess that and then to say, well, here's here's if we were to move this way, this is these are the kinds of costs that would and wouldn't show up on the accounting sheet of a firm. These are the costs that society would or wouldn't incur over these different time scales. Like here's how much this software would cost to buy today. But then over the longer time scale, maybe there's less opportunities for participation in skill development or research and that has a a vast cost as well. So how does that get addressed in that sort of techno tragedy of the commons? Yeah, I mean also I mean obviously we're not addressing this in the paper. Those are very deep questions and um I think it's yeah it's very challenging to to think about um like you know let's say you have a bunch of packages that are all doing something that is very similar as you point out. Is this necessarily wasteful? Maybe not because we can think about like this just as an evolutionary process, right? um like we need a couple of um projects or ideas out there and then some of them over time will survive and they will be selected by um you know whatever you want to call the best in this case maybe the one that just is in highest demand. Um but there or like there's some kind of lock in effect because people have used it for some things um initially and then um you know that's why we're why this particular package will be used uh all the time. Um so yeah so I think there's an evolutionary aspect uh to that question. Um but I I also agree on like sustainability of the environment and we need to find a way of especially once you know their packages have risen to prominence substantially whatever that means right there there may be some kind of threshold of uh demand. Then we have to think about um can can the community support this? Are there or can there be company efforts? Can there be new models um on supporting the the open-source software environment? Um because those packages that are used substantially, they are contributing substantially to the value um of open source software, not just on the individual side, but also uh from a company perspective, even though they're maybe not as engaged. And so that's one of the really the bigger picture questions that I see is also like how to pull in uh companies that are currently benefiting so much and um yeah like so that they are paying part of the cost but also getting something in return even maybe even more than what they're getting right now by um

contributing to this ecosystem in a in a in a way in a way where it stimulates simul innovation that is potentially beneficial for for them, right? And you can't ask a firm to maximize profits and um you know do engage in things for free but um there can be a profit motive in the long term and engagement and innovation and I think that's generally true for innovation. So yeah, so find figuring out those models um I think are is hugely important and you mentioned yeah I think initially you mentioned like services like that that we in the initial phases of um or early phases still um of open source software development right we in like I don't know by which measure if it's like uh 30 years or 50 years or whatever the a good measure is for That's that's hard to hard to say but um yeah so there are services like git and github and the version control kind of ideas out there but yeah and there may be new services like uh for example codeql where AI is uh substantially used to then u provide you with ideas of well here is maybe a security problem right and you can fix that and then you can use AI or you can you don't have to um to fix it um and so Those are additional tools that um became available and are becoming available and maybe some of them are more difficult for beginners than than others. Um but I could also see that some of them could rise to prominence. uh similarly but they of course we talk about those AI tools again this can come with pit pitfalls um as well when we have the whole environment flooded potentially by AI generated code um and we all know that that can um then the AI has potentially less training data that is not not unique anymore um but yeah that's that may be a different story but there are moving into the space of AI I guess there is um there are more and more kind of tools where a few of them may survive. You think about this evolutionary aspect again. There's this concept. Yeah. Yeah. Please. So there's this concept and I don't know if you're familiar with it. A lot of tech people like it. It's called technical debt where when you adopt something maybe you're locked into some solution and you and it costs you as an organization. So, I mean, you know, I wonder how the value of open source fits into that where you're losing value because you're locked into a suboptimal platform, but you're also there's an opportunity to gain value by maybe adopting working around it through open source or being able to modify your software. Yeah. Yeah, I think that's right. Um, right. So this technical debt issue with with a lock in problem that pro probably exist um is higher this kind of technical debt when you think about closed um form like closed software proprietary uh software that you're locked in and you maybe even have a harder time because you need um maintainers that are you know you have only a limited set of individuals that are familiar with this particular software while in for open source software you potentially have always new people that could come in and look at the software and um yeah you can

adopt it more easily. Um so you can switch more easily. Um fixes can be generated more quickly. Um just the whole idea of crowd sourcing. Um so I think there yeah generally technical debt as you would be as you are saying is um is going going to be lower with open source uh software. um you need to have the I guess like the right like still services around open-source software for developers to then actually engage in all of that uh type of work right um and I think there are companies out there are people out there that are doing exactly that you just have to you know work with the right people um but like I guess in terms of how that impacts the value of open source software um it's a good question I would I think it leads to a potential underestimate here again, right? Because it's just um so much more like e like you just have less of a of of a burden here. And um so so the value is automatically higher because you can you can switch more quickly from one innovation to another. But what exactly that value is that is uh hard for me to say or like how to measure that. Um but yeah, intuitively that would be um again contributing to the idea of underestimation. Yeah, it's a great point. Yeah, it reminds me of the the economic estimates like every window could be broken and then we would be adding value by just rebuilding. So that's the kind of rebuild approach which and then that that risk or like strategic value of like using an open schema or offsourcing or having the opportunity for people to offsource uh and then how to quantify like the value of optionality or of resilience to certain kinds of technical changes or like if you're relying on a proprietary software even for like a microscope in a lab or simulation toolkit and then the company slows down or goes out of business or or you know any number of other things. Whereas like if you have the source code it'll never get worse than it is. Maybe one of its dependencies could have like a future security issue but you'll always have that local function. And that's sort of what we need for the scientific reproducibility either way where like in the neuroiming uh community it there's a well-developed stack to talk about which versions of which software are used that that Bradley and colleagues have shared um previously and also Pedro writes in the live chat there is implicit trust given to companies that build in the open. So that's value for the companies or for the labs who are able to put out this useful thing. Uh Bradley if you have any other comments otherwise Manuel I will look forward to hearing like okay yeah what are your what are your next moves what are exciting directions and and how you're continuing this research or other projects? Um yeah sure. Well yeah thank you. Um yeah this is great. Um I really like this discussion. Um in terms of the value of open source software what I would like to see which is very difficult. I would love to just see a map or have a map of usage um and maybe not just usage but really then the value created um across

the world. I think that would be a you know ambitious endeavor that's very tough especially because you have to you know some of this information is proprietary and exactly in this instance here we were not able to get such granular information where we could where we were able to map everything to every region in the world but I think that would be a great descriptive starting point for an an addition for this value of open source software project and just generally um figuring out more information on usage across the world uh within companies um what are the which kind of packages are used when and where I think it's extremely valuable and important and yeah so I think that's on the value of open source side now I started a little bit talking about AI um get around that but so now the value of open source software one thing that we are pondering uh about is um what about the value of open source um or opensource AI if you like and we can you know think about what that concept actually is but um there's of course this a similar discussion that was that is around software generally right open world is proprietary which uh I think is not has been basically or is basically concluded almost where open source software one and um now a similar discussion exists in the space of AI, right? Should it be open versus closed? And uh so we would like to understand like what's the value of open-source AI relative to closed form alternatives? Um that would be you know in the spirit of this project. We would love to better understand that as well. But I think there's also a lot of work even before thinking about AI. Um yeah on on how to um yeah better understand the value maybe across industries even more. um we did this just for the externally facing data set. If we can really dig into an internally facing data set and are able to um show some of the estimates across industry, I think that would be fantastic. But again also that hinges on finding the right uh organizations, companies that are willing to share some of those that information that we need uh to then um engage in our value estimation. And another one is um you know sharing ju just the methodology broadly speaking and hopefully the methodology being adopted more widely even um you know let's say on GitHub or any kind of open source uh platform where people could get a rough sense of an like value estimate right and and this won't be perfect by any means but it's it's something that uh at least is a little bit more tangible uh so that would be great uh great tool Um yeah so I think that's what we're doing in terms of um better understanding the value of open source software we have also um other projects in the space of open source software um in particular we have a project that is more on generative AI and better understanding um GitHub copilot and how it affects the nature of work of individuals so that's that's I I think an exciting project which is not as you know closely tied to this project here but where we just try to see how work

patterns are changing when you have access to um this generative AI um coding tool and we see that people seem to engage in more coding that's u kind of project management activity and we see a lot of like kind of experimentation um out there a little bit less of exploration and can uh at the end of the day um yeah we see that people are engaging more autonomous and less collaborative work um because they you know like they can use the AI tool instead of um engaging with the community where um which we like to call a project management is really engaging with uh other developers on the issues board and then assigning other individuals and um yeah at the end of the day um in terms of experimentation new we find that people are engaging more um exposing themselves more to new languages that are worth more in the marketplace and we were able to use that to to put a color value kind of also very much like back of the envelope calculation to um the GitHub copilot uh coding AI which is a couple of uh billion uh dollars uh just for exposing yourself to new languages. So yeah, we always try to think about, you know, the value that is generated um for the community or when the community interacts um with tools with open source or with with tools that are intersecting with the open source uh software space. So yeah, I think that's super exciting and and happy to chat with anybody or also emails are more than welcome and any questions. Thank you. Thank you. I think maybe in the future we will hear more from your research and a scientific open-source metaanalysis Bradley andor other in uh the audience that could be very interesting to trace back some of those and and just talk about it from a scientific computing value chain rather than the industrial. Yeah, sure. Sounds good. Yeah, that's great to hear. Cool. Thank you. Appreciate it. See you next time. You too. Thank you so much. Thank you.